

Journal des évolutions — Wash Manager

Récapitulatif descriptif de tout ce qui a été ajouté au-delà de la v1 initiale.

Chaque bloc suit le même patron : ****modèle** → service → route → migration → tests → page**. Les identifiants d'action d'audit sont en code.

A. Rentabilité — Marge par forfait

But : connaître le coût réel et la marge de chaque forfait, par site.

- **Données** : `Produit.prix_unitaire` (coût d'achat/unité) ; recette de consommation existante (`ForfaitProduit.lavages_par_unite`).
- **Calcul** (`services/marge.py`) : coût produits d'un lavage
$$= \Sigma (\text{prix_unitaire} \div \text{lavages_par_unite}) ; \text{marge} = \text{prix} - \text{coût} ; \text{taux} = \text{marge}/\text{prix}.$$
- **API** : GET `/marges`, GET `/marges/{forfait_id}` (filtrables par site).
- **UI** : page **Marges** (prix, coût, marge, taux, détail par produit) ;
prix unitaire éditable dans l'Inventaire.
- **Tests** : 5 (calcul, sans recette, filtre site, liste, API).

B. Multilingue FR / AR

But : interface en français et en arabe, avec sens de lecture inversé.

- **Front** : `i18n/{fr,ar}.js` (mêmes clés), `context/I18nContext.jsx`
`(useI18n() → { t, lang, setLang, dir })`, `components/LangSwitcher.jsx`.
- L'arabe bascule `<html dir="rtl">` → **toute la mise en page passe à droite**.
- La langue est **persistée** (`localStorage`).
- **Doc** : `frontend/src/i18n/README.md`. Parité des clés vérifiée.

C. Clôture de caisse (rapport Z)

But : contrôler l'argent réellement encaissé chaque jour.

- **Données** : `ClotureCaisse` ; `Ticket.mode_paiement` (espèces/carte/autre)
et `Ticket.site_id` (renseignés à l'encaissement).
- **Service** (`services/cloture.py`) : agrégation par mode, écart de caisse
$$= \text{espèces comptées} - (\text{fond} + \text{espèces théoriques}), \text{ génération PDF}.$$
- **API** : GET `/cloture/aperçu`, POST `/cloture` (admin/manager, une par jour/site → 409 sinon), GET `/cloture`, GET `/cloture/{id}/pdf`.
- **UI** : page **Clôture de caisse** (aperçu par mode, comptage, écart en

direct, historique, PDF) ; sélecteur de mode de paiement à la Caisse.

- **Tests** : 5 + 4 de coordination (caisse→clôture, sécurité multi-site).

D. Santé des caméras (supervision)

But : repérer une caméra muette (angle mort de surveillance).

- **Données** : Camera (dernière vue, compteurs frames/events, file d'attente).
- **API** : POST /events/heartbeat (clé d'ingestion, upsert), GET /cameras
(calcule *en ligne / hors ligne* selon la fraîcheur du heartbeat, < ~90 s).
- **Edge** : EventClient.heartbeat() + service périodique Heartbeat
(1 signal/caméra) ; chaque CameraWorker compte ses frames/événements.
- **UI** : page **Caméras** (badges en ligne/hors ligne, silence, compteurs).
- **Tests** : 3 (backend) + 5 (edge).

E. Objectifs & écarts

But : fixer des cibles et voir chaque jour si elles sont atteintes.

- **Données** : Objectif (site, métrique, cible, sens min/max).
- **Métriques** : CA journalier, taux de conformité, lavages non facturés, véhicules traités.
- **Service** (services/objectifs.py) : valeur réelle du jour vs cible + écart.
- **API** : GET /objectifs/metriques, GET /objectifs, POST /objectifs
(upsert par site+métrique), DELETE, GET /objectifs/evaluation.
- **UI** : page **Objectifs** (cible vs réel, écart, badge Atteint/Manqué).
- **Tests** : 4.

F. Alertes (notifications temps réel)

But : passer d'un contrôle *consultatif* à *actif*.

- **Données** : Notification (historique + file d'envoi).
- **Déclencheurs** : anomalie *haute/critique* (dans
ingestion.finaliser_transaction) ; écart de caisse (route clôture).
- **Canaux** : email (SMTP) et WhatsApp (API), activés par la config ; sinon
statut « simulé » (enregistré, non transmis — rien n'est perdu).
- **Envoi différé** : notifier() enregistre en en_attente (non bloquant) ;
un **job du scheduler** (envoyer_notifications_en_attente, toutes les
alert_dispatch_interval_s) transmet et met à jour le statut → envoie/echec.

- **API** : GET /notifications, GET /notifications/canaux,
POST /notifications/test.
- **UI** : page **Alertes** (historique, sévérité, statut, bandeau mode simulé, bouton test).
- **Tests** : 8.

G. Journal d'audit (traçabilité)

But : savoir *qui* a fait *quoi*, *quand*, sur les actions à risque.

- **Données** : JournalAudit (snapshot email/rôle, action, cible, détails, site).
- **Actions tracées** : ticket.annuler, ticket.rapprocher,
forfait.modifier, forfait.supprimer, anomalie.resoudre, cloture.creer,
stock.mouvement, produit.supprimer.
- **API** : GET /audit (admin/manager, site-scopé, filtre par action).
- **UI** : page **Journal d'audit**.
- **Tests** : 5. *(Correctif au passage : POST /anomalies/{id}/resoudre renvoyait 500 sur un id inconnu → 404.)*

H. Export comptable (CSV)

But : fournir les chiffres au comptable.

- **API** : GET /export/recettes.csv (tickets encaissés, annulés exclus),
GET /export/clotures.csv — par période et par site, admin/manager. CSV
Excel FR (séparateur ; + BOM UTF-8).
- **UI** : page **Export comptable** (période + site, téléchargement authentifié).
- **Tests** : 3.

I. Pipeline IA (edge) — fiabilité & robustesse

1 Identification du laveur par gilet (bout en bout) : détection des

personnes (detector), association véhicule↔laveur (vest.bbox_la_plus_proche),
couleur canonique (vest.snap_couleur), émission d'un événement BADGE.
Rôle bay de l'orchestrateur ; palette de gilets **chargée dynamiquement**
du backend (GET /events/palette-gilets).

1 Corrélation multi-caméras : clé de suivi préfixée par la caméra

(camera_id:track_id) → plus de collisions ; topologie recommandée « une
caméra par baie » (rôle bay, couvre entrée+zones+sortie).

- 1 **File locale durable** (pipeline/outbox.py, SQLite) : aucun événement perdu en cas de coupure ; renvoi ordonné (FIFO) ; **flush périodique**.
- 1 **Idempotence** : EventIn.event_id (uuid) ; table EvenementTraite ; une re-livraison n'est pas appliquée deux fois.
- 1 **Robustesse capteur** : hystérésis + franchissement directionnel des lignes, anti-rebond des zones (moins de faux positifs).
- 1 **Correctifs** : sélection de la transaction *la plus récente* pour un track_id réutilisé ; LPR en erreur ne casse plus la boucle.
- **Tests IA** : passés de 7 à 43.

Récapitulatif technique

Nouvelles tables	cloture_caisse, cameras, objectifs, notifications, journal_audit, evenements_traites
Colonnes ajoutées	produits.prix_unitaire, tickets.mode_paiement, tickets.site_id, transactions.inventaire_consomme, employes.couleur_gilet
Migrations Alembic	chaînées de 7a6b54f8d20b à b7c8d9e0f1a2 (batch mode pour SQLite)
Tests	125 backend + 43 IA (départ ~44)
Langues	FR + AR (RTL)

Voir aussi : FONCTIONNALITES.md (catalogue), API.md (endpoints),
MODELE_DONNEES.md (tables), GUIDE_DEVELOPPEUR.md, GUIDE_UTILISATEUR.md.